

Mean Normalization

In linear regression, feature scaling helps the algorithm run faster. In recommender systems, a similar technique called **Mean Normalization** is used. It improves computational efficiency, but more importantly, it solves a critical issue regarding **new users** who haven't rated any movies yet.

The Problem: The "Cold Start" for New Users

Let's introduce a new user, **Eve** (User 5), to our dataset. Eve has just signed up and has not rated any movies yet.

- **The Issue:** Because Eve has no ratings ($r(i, j)$ is never 1), the cost function for her parameters does not have any data to fit.
- **The Result of Regularization:** The regularization term in the cost function tries to keep parameters small. Without data to counteract this, the algorithm will minimize Eve's parameters to zero:
 - $w^{(5)} = [0, 0]$
 - $b^{(5)} = 0$
- **The Prediction:** If we try to predict Eve's rating for *any* movie:
$$f(x) = w^{(5)} \cdot x^{(i)} + b^{(5)} = 0 \cdot x^{(i)} + 0 = 0$$
The system will predict that Eve hates every single movie (0 stars). This results in terrible recommendations.

The Solution: Mean Normalization

Mean normalization forces the "default" prediction for a new user to be the **average rating** of the movie, rather than 0.

Step 1: Calculate Movie Averages

Organize the ratings into a matrix (Y). For each movie (row), calculate the average rating given by users who *have* rated it. Store these in a vector μ .

- *Movie 1 Average:* 2.5
- *Movie 2 Average:* 2.5
- *Movie 3 Average:* 2.0
- ...and so on.

Step 2: Normalize the Data

Subtract the movie average (μ_i) from every existing rating in that row.

$$Y_{norm}^{(i,j)} = Y^{(i,j)} - \mu_i$$

- If User 1 rated Movie 1 a **5**, and the average is **2.5**, the new normalized rating is **2.5**.
- If User 1 rated Movie 4 a **0**, and the average is **2.25**, the new normalized rating is **-2.25**.

Step 3: Learn Parameters

Train the collaborative filtering algorithm using the **new, normalized values** (Y_{norm}) to learn w , b , and x .

Step 4: Make Predictions (Add the Mean Back)

When making a prediction, we calculate the dot product as usual, but we must **add the mean** (μ_i) **back** to get the final star rating.

$$Prediction = (w^{(j)} \cdot x^{(i)} + b^{(j)}) + \mu_i$$

Why This Fixes the Problem

Let's look at **Eve** (User 5) again using this new method.

1. Since she has no data, the regularization still results in $w^{(5)} = [0, 0]$ and $b^{(5)} = 0$.
2. **However**, the prediction formula has changed:
 $Prediction = (0 \cdot x^{(i)} + 0) + \mu_i = \mu_i$
3. **Result:** The system now predicts that Eve will rate a movie equal to the **average rating** that movie received from other users.
 - If a movie is generally liked (Avg 4.5), we predict Eve will give it a 4.5.
 - If a movie is generally disliked (Avg 1.0), we predict Eve will give it a 1.0.

This is a much more reasonable starting point than assuming 0 stars for everything.

Implementation Note: Rows vs. Columns

- **Normalize Rows (Movies):** Helps when you have a **new user**. (Recommended).
- **Normalize Columns (Users):** Helps when you have a **new movie** that no one has rated yet.
 - *Note:* Normalizing columns is less critical because if a movie has 0 ratings, the system shouldn't be recommending it to many people anyway. It is more important to give good recommendations to new users immediately.

Implementing Collaborative Filtering with TensorFlow

You are used to TensorFlow for Neural Networks, but it is also excellent for other machine learning algorithms because of one superpower: **Auto Diff (Automatic Differentiation)**.

The Power of Auto Diff

Normally, to use Gradient Descent, you need to manually calculate the partial derivatives of your cost function (using calculus).

- **Without TensorFlow:** You must do the math to find $\frac{\partial J}{\partial w}$, $\frac{\partial J}{\partial b}$, etc., and write code for it.
 - **With TensorFlow:** You just write the code to calculate the cost J . TensorFlow automatically figures out the derivatives for you.
-

How it Works: GradientTape

TensorFlow uses a tool called `tf.GradientTape` to record the operations that happen inside a block of code.

Basic Example (Simple Regression):

We want to minimize $J = (wx - 1)^2$.

```
1 w = tf.Variable(3.0) # Initialize parameter w
2 x = 1.0
3 y = 1.0
4 alpha = 0.01         # Learning rate
5
6 for i in range(30):
7     with tf.GradientTape() as tape:
8         # 1. Compute the cost J
9         fw = w * x
10        J = (fw - y)**2
11
12        # 2. Automatically calculate the derivative dJ/dw
13        grads = tape.gradient(J, w)
14
15        # 3. Update the parameter w using gradient descent
16        w.assign_sub(alpha * grads)
```

- `w.assign_sub(...)` is TensorFlow's way of doing $w = w - \dots$
-

Implementing Collaborative Filtering

For Collaborative Filtering, the cost function is complex (involving w , b , and x). Because this custom cost function doesn't fit into a standard Neural Network layer (like `Dense`), we can't use the simple `model.compile / model.fit` syntax.

Instead, we use a **Custom Training Loop** with `GradientTape`.

The Algorithm Structure:

1. **Define Optimizer:** Use a powerful optimizer like **Adam** instead of basic gradient descent.

```
1 | optimizer = keras.optimizers.Adam(learning_rate=1e-1)
```

2. **Training Loop:**

```
1 | for iter in range(200):
2 |     with tf.GradientTape() as tape:
3 |         # Calculate the full Collaborative Filtering Cost J
4 |         # (Using w, b, x, Y_norm, R, lambda, etc.)
5 |         cost_value = cofi_cost_func_v(X, W, b, Ynorm, R, num_users,
6 |                                     num_movies, lambda_)
7 |
8 |         # Calculate gradients for ALL parameters (X, W, b) simultaneously
9 |         grads = tape.gradient(cost_value, [X, W, b])
10 |
11 |        # Update parameters using Adam optimizer
12 |        optimizer.apply_gradients(zip(grads, [X, W, b]))
```

Why Do It This Way?

Since we are implementing a custom cost function that learns the features x (which are variables, not constant inputs), we need the flexibility of `GradientTape`. It allows TensorFlow to track gradients for x just as easily as it does for w and b .

Dataset: MovieLens

In the practice lab, you will use the real-world **MovieLens** dataset to build this system yourself. You will see how the algorithm learns to predict ratings for actual movies.

Finding Related Items 🔍

One of the most useful features of recommender systems is the ability to show a user "similar items" (e.g., "If you liked this book, you might also like..."). Collaborative filtering provides a natural way to do this.

How It Works

The algorithm learns a feature vector $x^{(i)}$ for every item i .

- While individual features $(x_1, x_2, \dots)$ are often hard to interpret (they may not explicitly represent "Action" or "Romance"), collectively they capture the mathematical "essence" of the item.

To find items similar to item i :

1. Look at the feature vector $x^{(i)}$.
 2. Search for another item k with a feature vector $x^{(k)}$ that is **closest** to $x^{(i)}$.
 3. Calculate the **squared distance** between the two vectors:
$$\text{distance} = \sum_{l=1}^n (x_l^{(k)} - x_l^{(i)})^2 = ||x^{(k)} - x^{(i)}||^2$$
 4. The items with the **smallest distance** are the most similar.
-

Limitations of Collaborative Filtering ⚠️

While powerful, basic collaborative filtering has two major weaknesses, primarily related to data availability.

1. The Cold Start Problem

This occurs when there is insufficient data to make accurate predictions.

- **New Items:** If a new movie is published and no one has rated it yet, the system has no data to learn its feature vector x , so it doesn't know how to rank or recommend it.
- **New Users:** If a new user joins and hasn't rated anything, the system doesn't know their preferences (w and b).
 - *Note:* Mean normalization helps with new users, but it is not a perfect solution.

2. Inability to Use Side Information

Collaborative filtering relies *only* on the interaction (ratings) between users and items. It has no natural way to incorporate "side information" (external context).

- **Item Features:** You might know a movie's genre, studio, budget, or cast, but standard

collaborative filtering cannot easily use this data.

- **User Features:** You might know a user's age, location, gender, or search history.
 - **Contextual Features:** Even technical details like the user's device (Mobile vs. Desktop) or browser (Chrome vs. Safari) can correlate with behavior, but this algorithm ignores them.
-

The Solution: Content-Based Filtering

To overcome the "Cold Start" problem and utilize "Side Information," we need a more advanced technique called **Content-Based Filtering**. This is a state-of-the-art technique used in many modern commercial applications.